AdaBoost: механика и интуиция

Продвинутое МО, ВШЭ, 2026 Как ошибки первого дерева направляют обучение следующих

Главная мысль. Деревья обучаются последовательно. После каждого раунда AdaBoost повышает веса объектов, на которых текущее дерево ошиблось. Следующее дерево старается исправить эти ошибки, а итоговый ответ учитывает голоса *всех* построенных деревьев.

1. Обозначения

$y_i \in \{-1, +1\}$	истинный класс объекта	$G_m(x)$	ответ дерева m из $\{-1, +1\}$
$w_i^{(m)}$	вес объекта перед раундом m	err_m	взвешенная ошибка дерева
α_m	сила голоса дерева	$F_m(x)$	сумма голосов первых m деревьев

2. Что происходит в одном раунде

0. Инициализация

Всем объектам даём одинаковый вес: $w_i^{(1)} = 1/n$, поэтому $\sum_i w_i^{(1)} = 1$.

1. Обучаем слабый классификатор

Часто это пенёк: дерево глубины 1. Оно ищет разбиение, при котором *сумма весов* ошибочных объектов минимальна. Вес задаёт цену ошибки, но не меняет истинную метку и не требует дублировать строку в данных.

2. Измеряем его качество

$$\mathrm{err}_m = \sum_{i=1}^{n} w_i^{(m)} \mathbf{1}\{y_i \neq G_m(x_i)\}.$$

В первом раунде с равными весами это доля ошибок. Потом важна уже *сумма их весов*: две ошибки могут стоить дороже многих верных ответов.

3. Назначаем дереву голос

$$\alpha_m = \frac{1}{2} \ln \frac{1 - \mathrm{err}_m}{\mathrm{err}_m}.$$

Чем меньше взвешенная ошибка, тем больше α_m: точному дереву доверяем сильнее. При $\mathrm{err}_m = 1/2$ его голос равен нулю.

4. Меняем важность объектов

$$w_i^{(m+1)} = \frac{w_i^{(m)} \exp[-\alpha_m y_i G_m(x_i)]}{Z_m}, \qquad \sum_i w_i^{(m+1)} = 1.$$

Вес правильно классифицированного объекта уменьшается, а вес объекта с ошибкой увеличивается. Z_m нормирует веса до суммы 1. Следующее дерево обучается на тех же объектах, но видит уже другие приоритеты.

5. Собираем ансамбль

$$F_M(x) = \sum_{m=1}^{M} \alpha_m G_m(x).$$

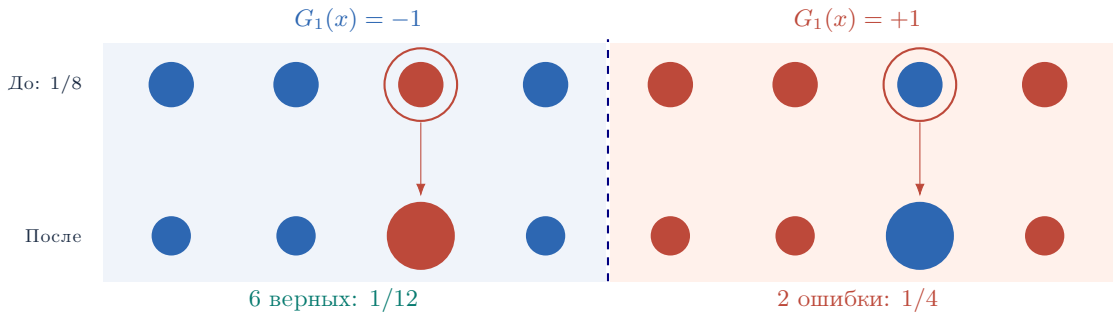
Каждое дерево отвечает -1 или $+1$; умножаем ответ на силу голоса и складываем. Если сумма положительна, предсказываем $+1$; если отрицательна, -1. При нуле голоса уравновесились и нужно правило разрешения ничьей. Старые деревья не переобучаются: к ним добавляют новые.

Почему это не обычное большинство. Пусть два дерева сказали $+1$ с голосами 0.2 и 0.2, а третье сказало -1 с голосом 1.0. Сумма $0.2 + 0.2 - 1.0 = -0.6$, поэтому ансамбль выберет -1, хотя за $+1$ голосовали два дерева из трёх.

Один раунд на восьми объектах

Как меняются приоритеты

Синие объекты имеют класс -1, красные $+1$. Первый пенёк разделил их вертикальной границей: слева предсказывает -1, справа $+1$. Изначально все восемь объектов имеют одинаковый вес: $1/8 = 0.125$. Пенёк ошибается на красном объекте слева и синем справа.



3. Первый раунд по шагам

1. Ошибка. Неправильных ответов два, и каждый пока весит $1/8$. Значит, $\mathrm{err}_1 = 1/8 + 1/8 = 1/4$.

2. Сила голоса. Суммарный вес верных ответов равен $3/4$, а ошибочных — $1/4$. Подставляем их отношение:

$$\alpha_1 = \frac{1}{2}\ln\frac{3/4}{1/4} = \frac{1}{2}\ln 3 \approx 0.55.$$

Дерево лучше случайного угадывания, поэтому его голос положителен.

3. Веса до нормировки. Каждый верный объект получает множитель $e^{-\alpha_1} = 1/\sqrt{3}$; каждый ошибочный получает $e^{\alpha_1} = \sqrt{3}$. Поэтому отношение веса ошибочного объекта к весу верного увеличивается в три раза. Цвет и правильный класс объектов при этом не меняются.

4. Нормировка. Чтобы сумма весов вновь стала равна 1, делим на $Z_1 = \sqrt{3}/2$. Получается:

$$\underbrace{w^{(2)}_{\text{верный}} = 1/12}_{\text{таких шесть}}, \qquad \underbrace{w^{(2)}_{\text{ошибка}} = 1/4}_{\text{таких два}}.$$

Проверка: $6 \cdot (1/12) + 2 \cdot (1/4) = 1$. На два объекта с ошибкой теперь приходится **половина общего веса**; ещё половина — на шесть верных. Один объект с ошибкой весит как три верных.

Следующий шаг. На новых весах прежнее дерево ошибается с суммарным весом $1/4 + 1/4 = 1/2$. Если перенести границу после второго объекта, новый пенёк исправит красный объект слева, но ошибётся на двух синих справа: их веса $1/12$ и $1/4$, вместе $1/3$. Этот кандидат лучше прежнего дерева на новых весах: $1/3 < 1/2$. Поэтому следующему дереву выгодно выбрать иную границу. Оно не обязано исправить все ошибки сразу; ансамбль складывает его голос с голосом первого дерева.

Отступ и связь с градиентным бустингом

Короткое объяснение после численного примера

4. Отступ: сколько уверенности в ответе

Отступ объекта равен $u_i = y_i F(x_i)$, где $F(x_i)$ — сумма взвешенных голосов. Если $u_i < 0$, ансамбль ошибся. Если $u_i > 0$, ответ верный; чем больше отступ, тем увереннее ансамбль.

В классическом AdaBoost используют экспоненциальную потерю e^{-u_i}. При $u_i = 2$ он около 0.14, а при $u_i = -2$ около 7.39. Поэтому трудные объекты получают больше внимания. Даже верный объект с отступом около нуля ещё не считается уверенно решённым.

5. Сходство и различия с gradient boosting trees

	AdaBoost здесь	Gradient boosting trees
Что видит новое дерево	веса объектов: ошибки становятся важнее	псевдоостатки, вычисленные из производной потери
Что добавляем	классификатор $G_m \in \{-1, +1\}$ с одним весом α_m	регрессионное дерево; значения листьев задают поправку
Потеря	экспоненциальная в классическом AdaBoost	выбирается под задачу

Общая идея: строим модели последовательно, и новая модель реагирует на слабые места уже собранного ансамбля. В AdaBoost это видно через веса объектов; в общем градиентном бустинге — через градиент выбранной функции потерь.

6. Итоговая формулировка

≪AdaBoost строит небольшие деревья одно за другим. Первое дерево ошиблось на двух из восьми одинаково важных объектов. После обновления каждый объект с ошибкой весит 1/4, а каждый верный — 1/12. Следующему дереву становится выгодно исправлять дорогие ошибки. В конце все деревья голосуют: ответ каждого умножается на силу его голоса, затем голоса складываются. Знак суммы определяет класс, а её модуль отражает уверенность ансамбля (это не вероятность)≫.

Оговорка. Если метка ошибочна, объект может оставаться трудным на многих раундах. AdaBoost будет повышать его важность и потому может быть чувствителен к шуму в метках.